

Examen - Desarrollo de Sistemas Móviles

Nombre: Angel Cesar

Apellidos: Obregon B laz

Código: 22200161

Fecha: 20/06/2020

2.- Analisis de Arquitectura

① El Patrón Repositorio Como SSOT

Esto se debe a que centraliza el acceso, al actuar como puente única de datos, esta aísla la lógica del negocio de las Tecnologías usadas para almacenar información. La coordinación se da de la siguiente manera:

- UI solicita datos al Repositorio sin enviar el origen
- Repositorio consulta primero la Room para una carga rápida y muestra un flujo de datos en cache
- A la vez, el Repositorio realiza una petición de red mediante Retrofit.
- Al recibir una respuesta exitosa de la API, el Repositorio en lugar de enviarlo directamente a la UI, actualiza la ROOM.
- UI reacciona a esta actualización, por lo que obtiene datos sin saber de la existencia de la API

② Corrutinas y Structured Concurrency

con ModelScope: Es un limitador de vida de corrutinas parte de las bibliotecas de desarrollo de Android, resulta muy útil en la gestión de memoria, ya que registra las corrutinas iniciadas y gestiona.

- Si el usuario cierra la pantalla antes de recibir respuesta
- El ciclo de vida de una vista termina y su `viewModel` se "destruye".
 - `ViewModelScope` cancela las corrutinas que se ejecutaban dentro
 - La petición que estaba en curso se interrumpe, ahorrando memoria, CPU y/o ancho de banda.

③ Flujos Reactivos (Cold vs Hot)

Flow: Es inactivo por defecto. Solo se ejecuta y emite datos cuando un collector está activo suscribiéndose a él, en caso de varios collectors, cada uno crea una ejecución desde 0.

StateFlow: Siempre activo en memoria, está en uso o no. Requiere valor inicial y conserva el último valor. A diferencia del Flow, varios collectors comparten la misma instancia.

Se prefiere usar `StateFlow` para UI porque esta necesita reacciones a cambios persistentes e inmediatos. Al volverse a conectar, como esta retiene el último valor no requiere la ejecución completa de la lógica desde 0, garantizando un uso fluido.

④ Inyección de Dependencias (DI) Manual

La DI manual usada consiste en crear un contenedor de dependencias centralizado para nuestra aplicación, en lugar de usar otras librerías. En la clase `Application`, instanciamos objetos

bases de la infraestructura para que permanezcan durante todo el ciclo de vida de nuestra app y puedan compartirse entre pantallas.

La palabra `by lazy` se utiliza para retrasar la creación de un objeto hasta que se usa por primera vez. En el caso de la B.D. o el Repositorio que son instancias pesadas se esperará a su uso para su creación, solo se crean cuando un componente solicita acceso a ellos, posterior a eso las llamadas devolverán la instancia ya creada, mejorando el tiempo de carga inicial de la aplicación.

3.- sección de completar

- ① El flujo de datos que emite automáticamente cada vez que una tabla de ROOM cambia se implementa usando el tipo `Flow`.
- ② El operador `Dispatchers.Default` permite ejecutar tareas pesadas fuera del Main Thread, específicamente optimizado para tareas de CPU como el procesamiento masivo de datos.
- ③ Para que un servicio en primer plano de ubicación sea válido en Android 10+ debe declararse en el manifiesto con el tipo `location`.

(4) La función *getApplication* de la clase *Application* es la que garantiza que los componentes compartan la misma instancia del repositorio mediante un "cast".

(5) El componente que permite transformar una API basada en callbacks antiguos en una función suspendible moderna se llama *suspendCancellableCoroutine*.

(6) el archivo de metadatos que describe la estructura de la app al sistema operativo se llama exactamente *AndroidManifest.xml*.

(7) En la arquitectura AOSP la capa que actúa como puente estándar entre el framework y los controladores se llama *Hardware Abstraction Layer (HAL)*.

(8) El operador de Flow que permite combinar múltiples puentes como GPS y sensores para crear un estado unificado es *combine*.

4.- Desarrollo de Código y Lógica Personalizada

(a)

```
data class Usuario (  
    val nombre: String,  
    val apellido: String,  
    val edad: Int?  
)
```


// (b)

```
fun calcularIdentidad() {
```

```
    val usuarioActual: Usuario? = Usuario(  
        nombre = "Angel Cesar",  
        apellidos = "Obregon Blaz",  
        edad = 21  
    )
```

// (c) y (e)

```
    val puntajeIdentidad = (usuarioActual?.nombre?.length  
        ?: 0) + (usuarioActual?.apellidos?.length ?: 0)
```

```
    val nombreCompleto = "${usuarioActual?.nombre ?: ""} ${  
        usuarioActual?.apellidos ?: ""}.trim()
```

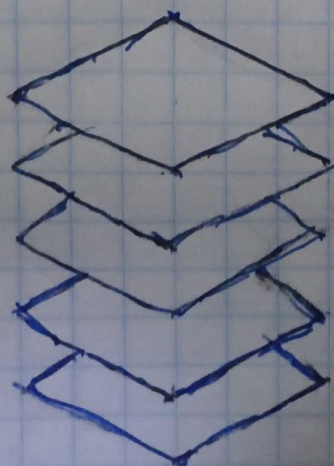
// (d)

```
    println("Usuario: $nombreCompleto, tu puntaje de identidad  
        es: $puntajeIdentidad")
```

}

5.- Arquitectura del AOSP Completa

(d) Grafico



1 Aplicaciones

2 Framework

3 Android Runtime y Librerías

4 Hardware Abstraction Layer (HAL)

5 Kernel Linux

② Identificación total

1 Aplicación : Capa Superficie donde se encuentran las aplicaciones por defecto y las instaladas por el usuario, interfaz que interactúa con el usuario final.

2 Framework Herramientas de los desarrolladores respecto al sistema operativo, en este nivel se encuentran los gestores como Activity Manager y otros, junto al sistema de vistas, los cuales controlan el ciclo de vida de la aplicación.

3 Android Runtime y Librerías : Entorno donde están tanto librerías nativas de C/C++ como las de compilador como el motor de renderizado.

4 HAL : Traductor entre software android y sensores / componentes del teléfono.

5 Kernel Linux : Núcleo del sistema donde están la RAM, CPU, energía y drivers más básicos.